

# **Assesment -1**

**Date:- 18/07/2026**

# **Session – A**

## **Session 1**

### **❖ Which development methodology would you recommend for this project, and why?**

#### **➤ Recommended Methodology: Agile**

#### **➤ Reason:**

This project has **frequently changing requirements**, and the client wants to **review working features every two weeks**. Agile works in short iterations (sprints), allowing the team to make changes quickly, deliver regular updates, and get continuous client feedback.

#### **Why Agile is suitable:**

- Requirements can change at any time.
- Working software is delivered every 2 weeks.
- Client feedback is included in every sprint.
- Problems are identified and fixed early.
- Faster delivery of useful features.

### **❖ Two risks if the team chooses the wrong methodology.**

#### **Difficulty handling changing requirements:**

- Waterfall requires all requirements to be finalized before development.
- the client changes requirements later, it causes delays, extra cost, and rework.

#### **Late client feedback:**

- In Waterfall, the client usually sees the final product near the end.
- If the product does not meet expectations, major changes are expensive and time-consuming.

## **Session – 2**

❖ **Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.**

- **Git Workflow:** Use the Feature Branch Workflow. Every developer should work on a separate branch, commit changes, push the branch, create a Pull Request, and merge into the main branch only after code review and testing.
- **Two Git Commands:**
  - 1. git checkout -b feature-name:** Creates a new feature branch and switches to it, allowing developers to work independently.
  - 2. git commit -m "message":** Saves changes to the local repository with a descriptive message.

## **Session – 3**

- ❖ You are building an online internship application form for a college placement portal. The form must collect: applicant name, email address, phone number, preferred domain (one of: Web, Data, Mobile, AI), and a resume file upload. The institution requires that no field be left empty and that the email field only accepts valid email formats — without writing any JavaScript.

| Field            | HTML Input Type   |
|------------------|-------------------|
| Student Name     | text              |
| Email Address    | email             |
| Phone Number     | tel               |
| Preferred Domain | select (dropdown) |
| Resume Upload    | file              |

❖ **Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.**

```
<div class="container">
  <div class="row">
    <div class="col-12 col-md-4">Article</div>
  </div>
</div>
```

- **Container** - page layout.
- **Row** - creates a row.
- **col-12** - 1 column on mobile.
- **col-md-4** - 3 columns on desktop ( $12 \div 4 = 3$ ).

❖ **Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop**

An array is better because it stores all 30 readings in one variable and makes looping easier.

**Two-step logic:**

1. Read all values and calculate the average.
2. Compare each value with the average and display above-average days.

**Why not one loop?**

The average is known only after all 30 readings are entered.

❖ **Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch**

The program crashes because an empty input contains only '\0'.  
Check before dereferencing:

**</>C**

```
if(*ptr == '\0')
```

This prevents accessing invalid data.

Pointer vs Array Traversal:

- **Pointer:** \*ptr, ptr++
- **Array:** str[i]

**Which is better?**

Pointer traversal makes it easier to detect an empty string before accessing memory.